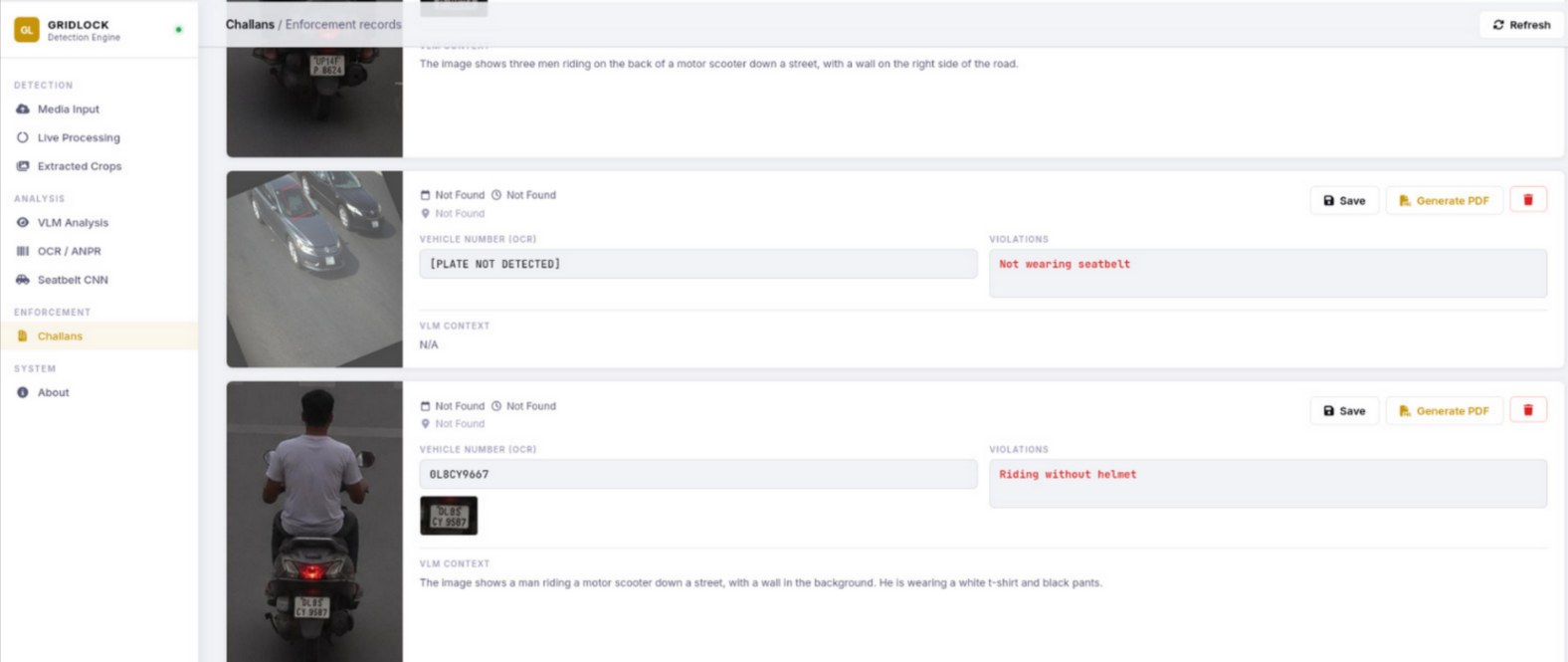




Automated Photo Identification and Classification for Traffic Violations Using Computer Vision

Gridlock Hackathon 2.0

Round 2 • Technical Report

Team GOPAKART

Manya Jain (Team Leader)

manya.jain@students.iiit.ac.in

Gopal Kataria

gopal.kataria@research.iiit.ac.in

K N Akthar Shaik

akthar.shaik@outlook.com

Jayant Gupta

jayg094c@gmail.com

1. Summary

Manual traffic enforcement is being outpaced by the scale of camera surveillance across Indian cities. Traffic cameras can continuously capture violations, but human review teams cannot consistently process such large volumes of footage, especially when the evidence comes from difficult real-world conditions such as motion blur, low light, shadows, crowded lanes, and unclear number plates. This project addresses that operational gap by building an automated computer-vision pipeline that converts raw traffic videos or images into structured, challan-ready violation records.

The system is designed around the actual enforcement workflow rather than only model prediction. Raw frames are first filtered and enhanced so that poor-quality footage becomes more usable for downstream analysis. A traffic-scene detection and reasoning layer then identifies violation-relevant evidence such as riders, helmets, seatbelts, vehicles, triple-riding cases, and license plates. The selected plate region is passed to an explainable OCR module that reads the registration number while preserving visual evidence for review. Finally, the system generates structured outputs such as the detected violation type, supporting evidence crop, license plate text, annotated image, and challan record, all of which are displayed through a dashboard for review.

The key strength of the project is that it avoids a single black-box verdict. Each stage produces evidence that can be inspected: the enhanced frame, object detections, plate crop, recognized characters, and generated challan data. This makes the pipeline suitable for real traffic enforcement, where decisions must be fast enough to scale but also transparent enough to be trusted. By combining preprocessing, violation reasoning, explainable license-plate reading, automated challan generation, and dashboard-based analytics, the system provides a practical foundation for faster, more consistent, and more accountable traffic violation processing.

2. Problem Analysis and Design Approach

2.1 Why a Modular Pipeline

The problem statement asks for a system that is simultaneously accurate, explainable to a human reviewer, and operable at scale across heterogeneous Indian road conditions. A single monolithic detector trained to output “violation / no violation” would be opaque, a traffic officer reviewing a generated challan needs to see why the system flagged a vehicle, not just a label. We therefore split the system into independently testable modules, each producing intermediate artifacts (enhanced frames, bounding boxes, captions, cropped plates, OCR text) that are retained as evidence rather than discarded after inference.

This modularity also matches how the team divided engineering work: one stream owns video preprocessing and the traffic-scene detector, and a second stream owns license-plate detection and OCR independently, with a defined interface (a cropped plate image in, registration text out) so the two can be developed, tested, and benchmarked in isolation before integration.

2.2 System Overview

3. Executive Summary & Project Purpose

GRIDLOCK2 is a hackathon-grade, AI-powered urban traffic management platform split into two core subsystems: a **Challans Dashboard** (traffic violation detection and e-challan generation) and a **StreetPark Ledger** (smart street parking management). Together, they address three major urban pain points-traffic law enforcement, parking fraud, and revenue leakage-using computer vision, deep learning, and a tamper-evident audit ledger.

@IX@ Subsystem Goal

Challans Dashboard Detect helmet, seatbelt, triple-riding, and stop-line violations from photos/video; auto-generate printable e-challans with ANPR plate text and GPS location.

StreetPark Ledger Record vehicle check-in/check-out via ANPR, calculate time-of-day dynamic fees, accept UPI/cash payments, escalate unpaid exits to government challan, and maintain an immutable hash-chained audit trail.

Manual traffic enforcement is being outpaced by the scale of camera surveillance across Indian cities. Traffic cameras continuously capture violations, but human teams cannot consistently process the volume of footage, especially under difficult conditions like motion blur, low light, and unclear plates. This project addresses that operational gap with an automated computer-vision pipeline, compiled into a working Flask application, converting raw footage into structured, challan-ready records reviewable through a browser dashboard.

The system is designed around the actual enforcement workflow rather than just model prediction. Raw frames are filtered, enhanced, and passed to a traffic-scene detection and reasoning layer. Explainable OCR reads the registration number, preserving visual evidence. Finally, the system generates structured outputs (detected violation, evidence crops, plate text, annotated images, and challan files). By combining preprocessing, multi-class violation reasoning, explainable ANPR, automated challan generation, and a live review dashboard, the system provides a practical, running foundation for transparent and accountable traffic violation processing.

4. System Architecture & Directory Structure

4.1 Directory Structure

The system architecture mirrors the modularity of the codebase, which is divided between the Flask-based Challans Dashboard and the FastAPI/Streamlit-based StreetPark Ledger.

```
Submission/
  Challans Dashboard/      # Traffic violation detection + e-challan system
    app.py                # Flask server + async ML worker
    pipeline_core.py      # Unified YOLO+VLM inference engine
    seatbelt_crop_classifier.py # ResNet-based binary seatbelt CNN trainer
    stop_line_detection.py # Florence-2 VLM stop-line detection
    video_core.py         # Frame extraction + quality filter
    index.html            # Single-page upload/review dashboard UI
    models/               # YOLO .pt weights (traffic, ANPR, vehicle)
    data/                 # uploads/, challans/, extracted/ (runtime)

  parking_ecosystem/      # StreetPark Ledger
    app.py                # Streamlit Driver Dashboard
    attendant_app.py      # Streamlit Attendant Portal + E-Challan Panel
    api.py                # FastAPI REST endpoints (public)
    db.py                 # SQLite schema, fee engine, audit chain
```

```

vision_node.py          # ANPR CLI node (YOLO11 + EasyOCR fallback)
parking_ledger.sqlite3  # Live SQLite database
modules/
  razorpay_integration.py # Razorpay checkout, webhook, links
  scheduler.py           # APScheduler jobs (overstay, blacklist)
sample_challan.pdf      # Output example

```

4.2 Execution Flow: Challans Dashboard

The system splits independently testable modules to retain artifacts rather than discarding them after inference.

Upload (image/video)

Flask API (/api/upload)

[async, single background thread]
Background ML Worker

```

Vehicle YOLO (yolo_vehicle_best.pt)  crop each detected vehicle
Traffic YOLO (traffic_yolo_best.pt)  classify: no_helmet / bad_helmet /
                                         no_seatbelt / triple_riding
Florence-2 VLM (optional fallback)  free-text caption for ambiguity
ANPR YOLO (anpr_yolo_best.pt)       character-level plate detection
GPS/EXIF extractor  Nominatim reverse geocoding

```

write_challan() → output/challan_<UID>/ with annotated JPEG + JSON

index.html fetches /api/results → live challan gallery

4.3 Execution Flow: StreetPark Ledger

Camera / CLI (vision_node.py --plate <XX00XX0000>)

record_detection() [db.py]

```

first sighting → CHECK_IN event (new session ACTIVE)
second sighting → CHECK_OUT event (session EXITED_UNPAID or CLEARED)

```

calculate_fee_dynamic() ← pricing_rules table (per-lot, per-hour, per-DOW)

Driver Dashboard	Attendant Portal
(Streamlit app.py)	(Streamlit attendant_app.py)

clear_payment() → session CLEARED + audit event

[grace period exceeded]

escalate_overdue() → challan_id issued, fine_rupees added

5. Module-Level Architecture: Challans Dashboard

5.1 Module A - Video Preprocessing

Module A prepares raw traffic footage for reliable downstream detection. The preprocessing pipeline reads the input video using `cv2.VideoCapture` and samples frames at controlled time intervals (1.5 seconds in the compiled application) instead of processing every frame. Each sampled frame is checked for sharpness using Laplacian variance (variance below 5.0 drops severely corrupted frames).

Accepted frames are then resized and enhanced. Temporal accumulation reduces frame-to-frame noise. Local contrast is improved using CLAHE on the LAB color space, especially useful for Indian road scenes with mixed shadows and bright exposure. Laplacian-based sharpening strengthens edges around vehicles and plates. This is handled centrally in `video_core.py`.

5.2 Module B - Traffic Violation Detection and Reasoning

Module B is the decision-making stage, heavily reliant on `pipeline_core.py`. It uses a custom-trained YOLO detector to locate violation-relevant objects (riders, helmets, seatbelts). Raw class names from different label sources are normalized so dataset-specific variants map to a canonical vocabulary. A practical false-positive filter suppresses `no_helmet` detections if they overlap a co-located `helmet` box.

In parallel, Florence-2 (a vision-language model) generates semantic descriptions. It acts strictly as fallback reasoning-YOLO direct violations are authoritative, and Florence-2 never overrides a high-confidence YOLO violation.

A separate path handles stop-line violations, using a manually calibrated virtual stop line and IoU-based tracking across frames to detect crossing events during red-light intervals.

5.3 Module C - License Plate Detection and OCR (ANPR)

Module C extracts the registration number using a custom YOLO character detector (`anpr_yolo_best.pt`). The compiled pipeline crops the plate region with padding and runs the detector across a sweep of confidence thresholds (0.35 down to 0.10). Characters are grouped vertically for two-line plates, sorted left-to-right, and concatenated. State-code correction and invalid-character removal ensure high accuracy for Indian layouts. This method shows exactly where each character was read from, making it highly auditable.

5.4 Module D - Application Layer and Review Dashboard

Module D makes the system operable. A compiled Flask backend exposes `/api/upload` that queues media into a background worker. The dashboard (`UI/index.html`) features a single-page interface with sections for media upload, live processing status, Extracted Crops, VLM Analysis, OCR/ANPR, Seatbelt CNN, and Challans. A reviewing officer can inspect every piece of supporting evidence and correct the OCR plate text or violation label directly in the browser.

6. Representative Evidence Outputs

The following figures show the main evidence artifacts produced by the system.



Original frame



Enhanced frame

Figure 1: Representative preprocessing comparison. Local contrast is improved and sharpened.



Input frame



YOLO detection output

Figure 2: Traffic-scene detection output showing YOLO detections for riders, helmets, and plates.

Predicted Plate: UP14FP8624



Figure 3: OCR detection evidence showing the predicted license plate text with character-level bounding boxes and confidence scores.

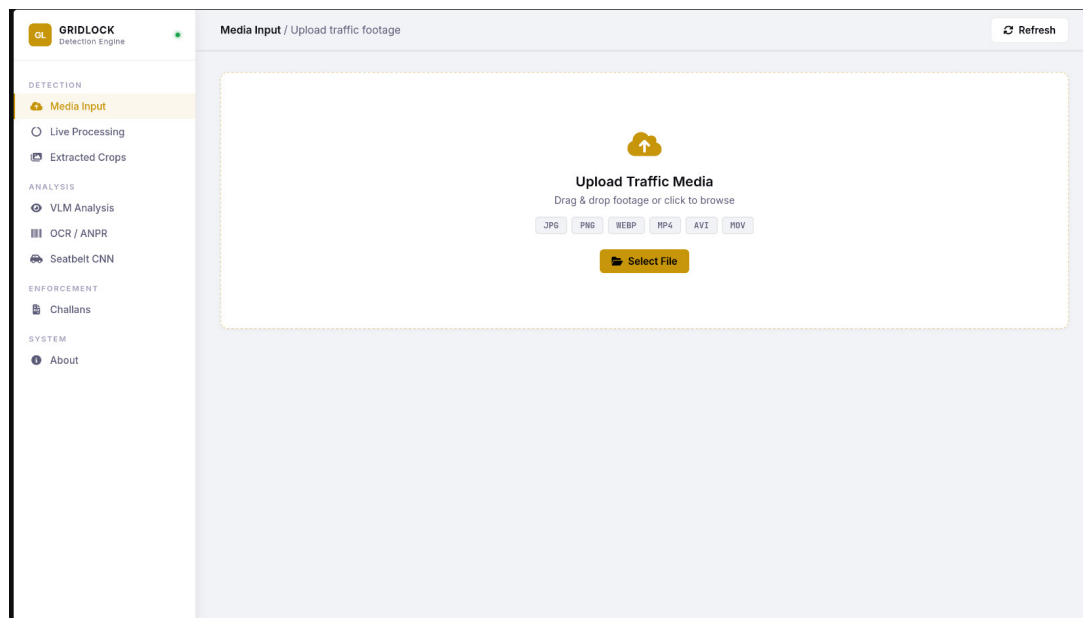


Figure 4: The Media Input screen of the active dashboard, showing the upload entry point and navigation structure.

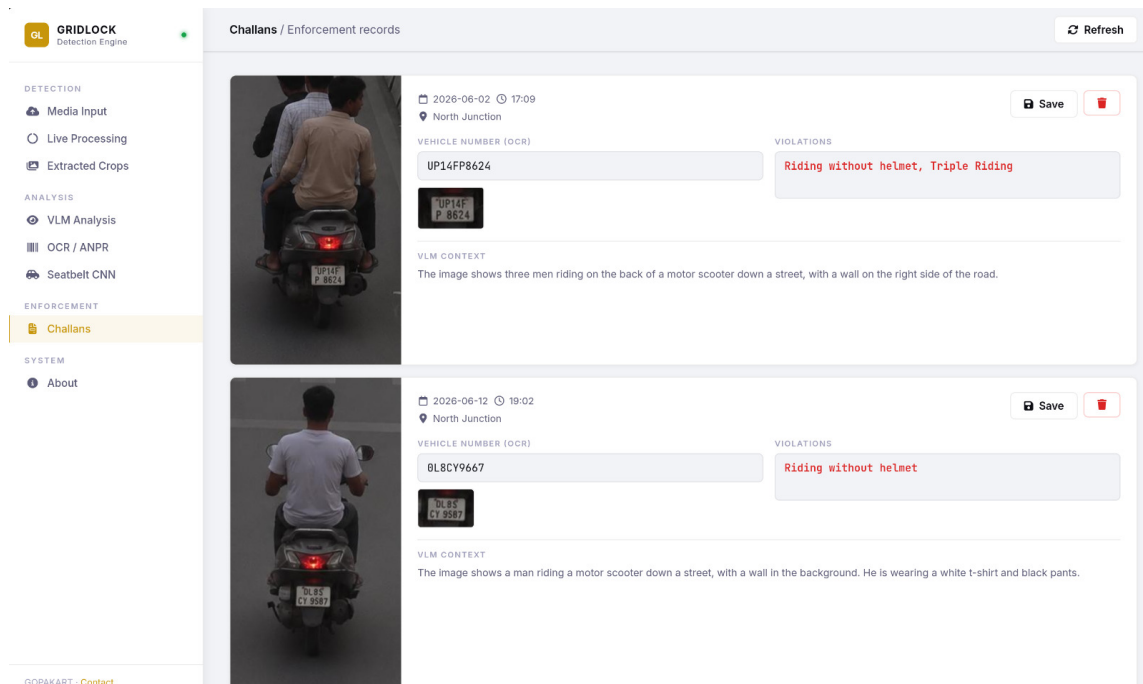


Figure 5: Real challan records from the review screen. Demonstrates accurate triple-riding detection and an explicit OCR misread that is visually auditable by an officer.

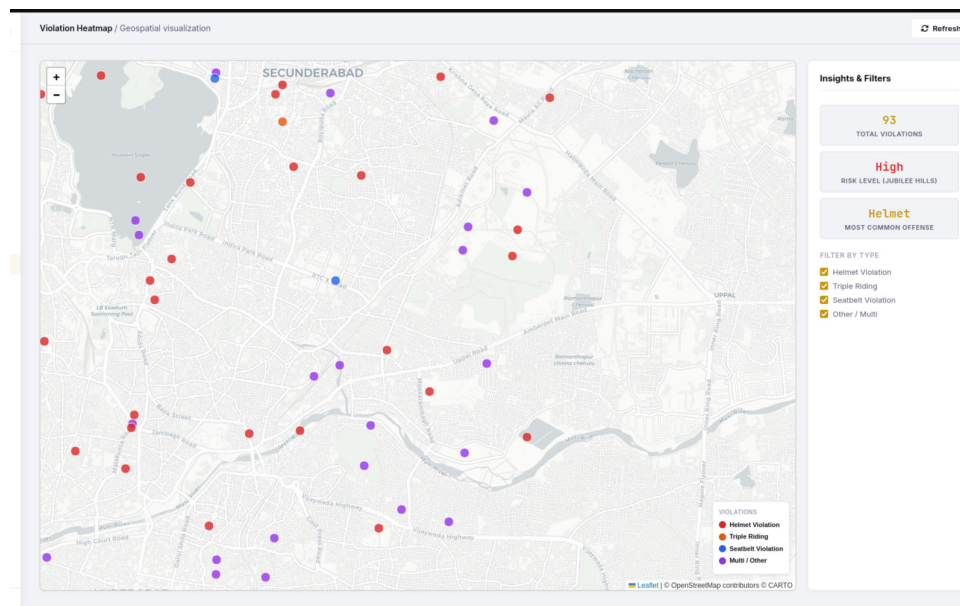
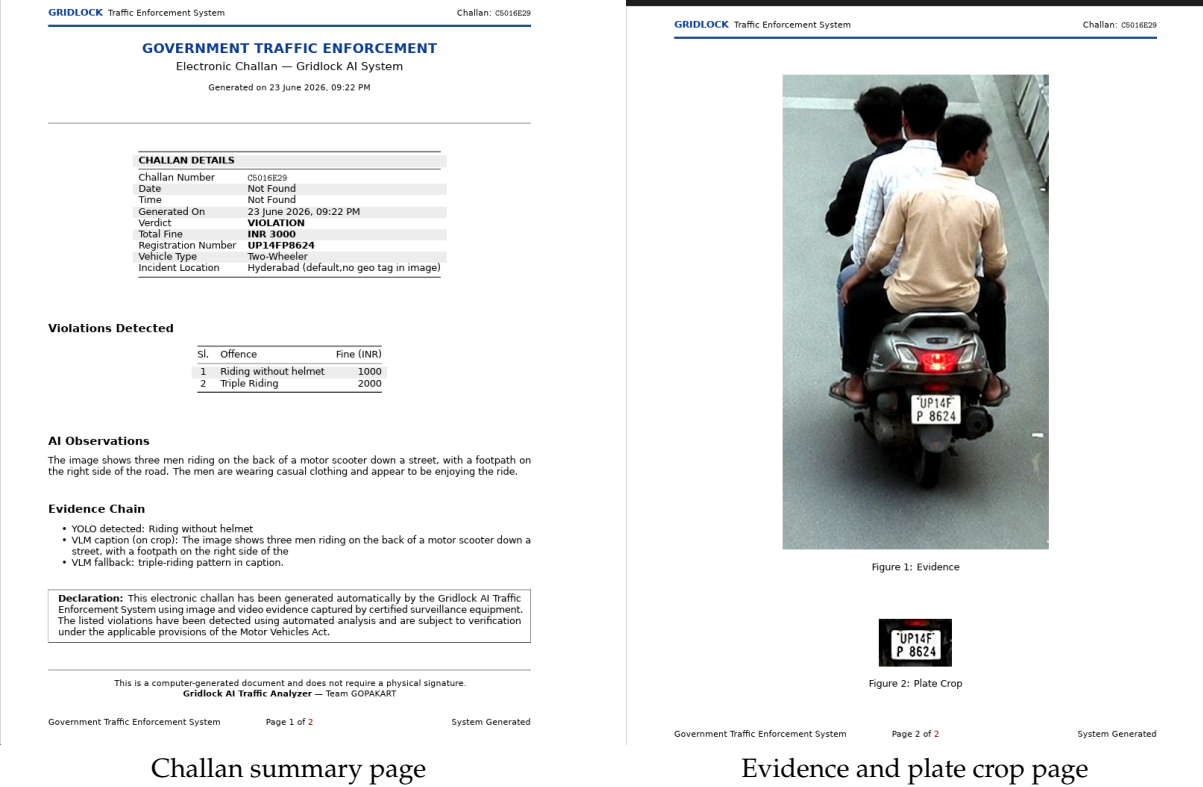


Figure 6: Violation heatmap dashboard showing geospatial distribution of detected traffic violations across the city map.



7.3 MobileNetV3-Small for Seatbelt Verification

Used as a dedicated binary crop classifier operating on localized occupant regions. It achieves strong accuracy-to-latency tradeoffs suitable for dashboard-time inference.

7.4 IoU Tracking for Stop-Line Violations

Stop-line violation relies on YOLOv8n's COCO vehicle classes and a lightweight IoU-based tracker to associate detections across frames and compute which side of a manually calibrated virtual line each vehicle occupies.

8. Implemented Violation Coverage

The system implements four of the seven named violation classes:

- **Helmet Non-Compliance:** Separates clean compliance scenes from `no_helmet` or `bad_helmet` cases. It is the most frequently triggered violation in local runs.
- **Seatbelt Non-Compliance:** Handled through the direct YOLO `no_seatbelt` class with a CNN crop classifier available as a secondary verification path.
- **Triple Riding:** Normalized YOLO class mapping combined with Florence-2 fallback reasoning (e.g., "three riders") when standard thresholds fail due to occlusion.
- **Stop-Line & Red-Light Violation (Prototype):** Tracks vehicles across frames against a manually calibrated virtual stop line. Flags crossings only if they occur during a manually supplied red-light interval. Integrating automatic signal-state detection is the next immediate step.

9. Results and Evidence Summary

The compiled system produces enhanced frames, tracked stop-line crossings, OCR visualizations, and a working dashboard.

Component	Precision	Recall	F1	mAP50	mAP50-95
ANPR character detector	0.9826	1.0000	≈0.9912	0.9950	0.9728

Table 1: ANPR character detector metrics. OCR text-reconstruction exact match on the corresponding 17-image test split was 1.0000 (note: evaluated in `final_artifact` mode).

Artifact (from UI/data/extracted, 25 records)	Count
Violation verdicts	18
Clean verdicts	7
Riding-without-helmet events	13
Not-wearing-seatbelt events	5
Triple-riding events	1
Raw OCR plate fields present	24
Valid-looking Indian registration strings	6
Non-detected plate placeholders	11

Table 2: Artifact counts from locally generated extracted records.

Artifact (from UI/data/challans, 57 records)	Count
Violation records	36
Clean records	21
Riding-without-helmet records	29
Not-wearing-seatbelt records	5
Triple-riding records	4
Valid-looking Indian registration strings	10
Plate-not-detected placeholders ([PLATE NOT DETECTED])	18
Mean / median OCR latency	188.44 ms / 174.33 ms

Table 3: Artifact counts from locally generated challan records.

10. Technologies, Challenges & Future Improvements

@IX@ Challenge Design Decision

Multiple YOLO Vocabularies `norm_class_name()` with a 30+ alias lookup table maps all variants to a canonical set before downstream logic.

Audit Tamper-Resistance SHA-256 hash chain in `audit_events`: each event stores `previous_hash`, making retroactive edits detectable.

Dynamic Parking Pricing Fee computed hour-by-hour against `pricing_rules` (lot $\times$ DOW $\times$ hour band) enabling peak/off-peak pricing.

GPU vs CPU Deployment Florence-2 and YOLO models auto-detect CUDA; fallback to float32. System degrades gracefully.

Video Frame Quality Two-tier Laplacian filter: hard-drop below score 5, soft-restore (CLAHE + brightness) between 5-80.

10.1 Future Improvements Roadmap

- **Real-time video streaming:** Replace frame-polling with WebSocket/RTSP for live CCTV feeds.
- **Distributed Inference:** Move ML workers to a separate GPU microservice behind Redis/Kafka for horizontal scaling.
- **Government API Integration:** Replace simulated challan IDs with actual Vaan/Parivahan API calls for enforcement.
- **Signal-State Fusion:** Integrate traffic-light state recognition (red/amber/green classifier) directly into the stop-line tracking module.
- **Mobile ANPR App:** Wrap `vision_node.py` in a REST container so traffic wardens can use smartphones as field nodes.

11. Conclusion

We set out to build a system that takes in raw traffic photographic evidence and produces structured, reviewable violation records alongside a robust parking management ledger. This is no longer a set of isolated notebooks; it is a compiled, working application. It features video preprocessing tuned for degraded conditions, a fused detector-plus-VLM violation reasoning layer covering four violation classes, an explainable ANPR module, and a Flask backend with a live dashboard.

Our roadmap is scoped to close the highest-value gaps first: independent benchmarking of the violation detector, fixing the plate-crop resolution bottleneck, correcting hardcoded deployment paths, and adding automatic signal-state detection. A system that executes these components end-to-end with running code and real generated evidence offers immediate practical value as a transparent, auditable enforcement platform.

12. References

References

- [1] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of the IEEE CVPR, 2016.
- [2] Ultralytics. *Ultralytics YOLO11 Documentation*. 2024.
- [3] Bin Xiao et al. *Florence-2: Advancing a Unified Representation for a Variety of Vision Tasks*. arXiv preprint arXiv:2311.06242, 2023.
- [4] Satadal Saha. *A Review on Automatic License Plate Recognition System*. arXiv preprint arXiv:1902.09385, 2019.
- [5] Rayson Laroca et al. *A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector*. arXiv preprint arXiv:1802.09567, 2018.
- [6] Andrew Howard et al. *Searching for MobileNetV3*. Proceedings of the IEEE/CVF ICCV, 2019.